BECKER CORP V8

DOCUMENTAÇÃO TÉCNICA DO SISTEMA

Gerado em: 31/01/2026, 13:43:01

Abrangência: Backend, Frontend, Proxy, Banco de Dados e Scripts.

1. BACKEND PRINCIPAL (API Core - Porta 6778)

Arquivo: server.ts

Entry Point: Servidor Express e Rotas Principais

```
    import express from 'express';
    import cors from 'cors';
    import https from 'https';
    import fs from 'fs';
    import authRoutes from './modules/auth/routes';
    import dashboardRoutes from './modules/dashboard/routes';
    import serverRoutes from './modules/server/routes';
    import backupRoutes from './modules/backups/routes';
    import controlRoutes from './modules/control/routes';
    import { startMonitor } from './modules/control/monitor';

    const app = express();
    const PORT = 6778;

    app.use(cors());
    app.use(express.json());

    // ROTAS
    app.use('/api/auth', authRoutes);
    app.use('/api/dashboard', dashboardRoutes);
    app.use('/api/server', serverRoutes);
    app.use('/api/backups', backupRoutes);
    app.use('/api/control', controlRoutes); // <--- ROTA DO CONTROLE

    app.get('/api/ping', (req, res) => res.json({ msg: 'Pong HTTPS' }));

    // INICIA IA SENTINELA
    startMonitor();

    try {
        const options = {
            key: fs.readFileSync('/etc/letsencrypt/live/console.jacarezinho.cloud/privkey.pem'),
            cert: fs.readFileSync('/etc/letsencrypt/live/console.jacarezinho.cloud/fullchain.pem')
        };
        https.createServer(options, app).listen(PORT, '0.0.0.0', () => {
            console.log(`>>> BACKEND ONLINE: ${PORT}`);
        });
    } catch (e) { console.error("SSL Error:", e); }
```

Arquivo: routes.ts

Módulo Proxy (Rotas Antigas/Compatibilidade)

```
    import { Router } from 'express';
    import { execCmd } from '../../utils/sys';
    import { Pool } from 'pg';
    const router = Router();
    const pool = new Pool({ connectionString: 'postgres://postgres:becker_admin_secure@localhost:5432/
controlebeckercorp_v8' });

    const CMD_CTL = "sudo /bin/systemctl";
    const CMD_HTPASSWD = "sudo /usr/bin/htpasswd";
    const SCRIPT_PANIC_ON = "sudo /opt/controlebeckercorp-v8/scripts/panic_on.sh";
    const SCRIPT_PANIC_OFF = "sudo /opt/controlebeckercorp-v8/scripts/panic_off.sh";

    // --- SYNC ---
    const syncSystem = async () => {
        try {
            const vips = await pool.query("SELECT ip FROM proxy_vips");
            const vipList = vips.rows.length > 0 ? vips.rows.map(r => r.ip).join('\n') : '';
            await execCmd(`echo "${vipList}" | sudo tee /etc/squid/vips.acl`);

            const nets = await pool.query("SELECT iface_name FROM net_interface_config WHERE
squid_enabled = true");
            const map: any = { 'docker':'172.16.0.0/12', 'vpn':'10.0.0.0/8',
'enp6s0.10':'192.168.10.0/24', 'enp6s0.30':'192.168.30.0/24', 'enp6s0.40':'192.168.40.0/24',
'enp6s0.50':'192.168.50.0/24', 'enp6s0.70':'192.168.70.0/24' };
            let allowed: string[] = [];
            for(const row of nets.rows) { if(map[row.iface_name]) allowed.push(map[row.iface_name]); }

            const netContent = allowed.length > 0 ? allowed.join('\n') : '';
            await execCmd(`echo "${netContent}" | sudo tee /etc/squid/allowed_nets.acl`);
            await execCmd(`${CMD_CTL} reload squid`);
```

```
        } catch (e) { console.error(e); }
    };

    // --- ROTAS ---
    router.post('/panic', async (req, res) => {
        const { action } = req.body;
        try {
            if (action === 'activate') {
                await execCmd(SCRIPT_PANIC_ON);
                res.json({ success: true, status: 'panic' });
            } else {
                await execCmd(SCRIPT_PANIC_OFF);
                res.json({ success: true, status: 'running' });
            }
        } catch (e) { res.status(500).json({ error: "Erro Panic" }); }
    });

    router.get('/interfaces', async (req, res) => {
        try { const r = await pool.query("SELECT * FROM net_interface_config ORDER BY iface_name");
res.json(r.rows.map(row => ({ name: row.iface_name, label: row.alias, enabled: row.squid_enabled, type:
row.type || 'vlan', subnet: 'Monitorado' }))); } catch { res.json([]); }
    });
    router.post('/interfaces/toggle', async (req, res) => { try { await pool.query("UPDATE
net_interface_config SET squid_enabled = $2 WHERE iface_name = $1", [req.body.iface,
req.body.enabled]); syncSystem(); res.json({success:true}); } catch
{ res.status(500).json({error:"DB"}); }});

    router.get('/users', async (req, res) => { try { const r = await pool.query("SELECT * FROM
proxy_users ORDER BY id DESC"); res.json(r.rows); } catch { res.json([]); } });
    router.post('/users', async (req, res) => {
        const { username, password } = req.body;
        try {
            await pool.query("INSERT INTO proxy_users (username, term_accepted) VALUES ($1, true) ON
CONFLICT (username) DO NOTHING", [username]);
            if(password) execCmd(`${CMD_HTPASSWD} -b -c /etc/squid/passwd ${username} ${password} 2>/
dev/null || ${CMD_HTPASSWD} -b /etc/squid/passwd ${username} ${password}`);
            res.json({ success: true });
        } catch { res.status(500).json({ error: "Erro User" }); }
    });
    router.post('/users/delete', async (req, res) => { await pool.query("DELETE FROM proxy_users WHERE
username=$1", [req.body.username]); execCmd(`${CMD_HTPASSWD} -D /etc/squid/passwd ${req.body.username}
`).catch(()=>{}); res.json({success:true}); });

    router.get('/vips', async (req, res) => { try { const r = await pool.query("SELECT * FROM
proxy_vips ORDER BY id DESC"); res.json(r.rows); } catch { res.json([]); } });
    router.post('/vips', async (req, res) => { await pool.query("INSERT INTO proxy_vips (ip,
description) VALUES ($1, $2)", [req.body.ip, req.body.desc]); syncSystem(); res.json({ success:
true }); });
    router.post('/vips/delete', async (req, res) => { await pool.query("DELETE FROM proxy_vips WHERE
id=$1", [req.body.id]); syncSystem(); res.json({ success: true }); });

    // LIMIT 50 PARA AUDITORIA OTIMIZADA
    router.get('/logs', async (req, res) => { try { const r = await pool.query("SELECT * FROM
proxy_audit_log ORDER BY timestamp DESC LIMIT 50"); res.json(r.rows); } catch { res.json([]); } });
    router.get('/status', async (req, res) => { try { const r = await execCmd(`${CMD_CTL} is-active
squid || echo inactive`); res.json({ status: r.trim() === 'active' ? 'running' : 'stopped' }); } catch
{ res.json({status:'stopped'}); } });

    export default router;
```

Arquivo: monitor.ts

IA Sentinela: Lógica de Monitoramento

```
    import { execCmd } from '../../utils/sys';
    import { sendAlert } from '../../utils/mailer';
    import { Pool } from 'pg';
    import * as os from 'os';

    const pool = new Pool({ connectionString: 'postgres://postgres:becker_admin_secure@localhost:5432/
controlebeckercorp_v8' });

    const CRITICAL_SERVICES = [
        'squid', 'postgresql', 'nginx', 'ufw', 'ssh',
        'fail2ban', 'wg-quick@wg0', 'isc-dhcp-server', 'smbd', 'unbound'
    ];

    const CHECK_INTERVAL = 10 * 60 * 1000;    // 10 Minutos
    const ALERT_REPEAT_TIME = 3 * 60 * 60 * 1000; // 3 Horas
    let lastHwAlert = 0;

    export const startMonitor = async () => {
```

```
        console.log(`>>> [IA SENTINEL] Vigilncia Ativa.`);

        // Executa imediatamente ao iniciar
        checkServices();

        setInterval(checkServices, CHECK_INTERVAL);
    };

    async function checkServices() {
        // A. Servios
        for (const svc of CRITICAL_SERVICES) {
            try {
                let currentStatus = 'stopped';
                try {
                    const statusRaw = await execCmd(`systemctl is-active ${svc}`);
                    currentStatus = statusRaw.trim() === 'active' ? 'running' : 'stopped';
                } catch {}

                const dbRes = await pool.query("SELECT last_status, last_alert_sent FROM
sys_service_status WHERE service_name=$1", [svc]);
                const memory = dbRes.rows[0] || {};
                const lastStatus = memory.last_status || 'unknown';
                const lastAlertTime = memory.last_alert_sent ? new
Date(memory.last_alert_sent).getTime() : 0;
                const timeSinceAlert = Date.now() - lastAlertTime;

                let shouldAlert = false;
                let reason = "";

                if (lastStatus === 'running' && currentStatus === 'stopped') {
                    shouldAlert = true; reason = "Queda Imediata";
                } else if (currentStatus === 'stopped' && timeSinceAlert >= ALERT_REPEAT_TIME) {
                    shouldAlert = true; reason = "Inativo Persistente";
                }

                if (shouldAlert) {
                    console.log(`[IA SENTINEL]  ALERTA: ${svc}`);
                    await sendAlert(`FALHA CRTICA: ${svc}`, `Servio PARADO.\nMotivo: ${reason}`);
                    await pool.query("INSERT INTO sys_service_status (service_name, last_status,
last_checked, last_alert_sent) VALUES ($1, $2, NOW(), NOW()) ON CONFLICT (service_name) DO UPDATE SET
last_status=$2, last_checked=NOW(), last_alert_sent=NOW()", [svc, currentStatus]);
                } else {
                    await pool.query("INSERT INTO sys_service_status (service_name, last_status,
last_checked) VALUES ($1, $2, NOW()) ON CONFLICT (service_name) DO UPDATE SET last_status=$2,
last_checked=NOW()", [svc, currentStatus]);
                }
            } catch (e) { console.error(`[IA ERROR] ${svc}`, e); }
        }

        // B. Hardware
        const now = Date.now();
        if (now - lastHwAlert > ALERT_REPEAT_TIME) {
            try {
                const totalMem = os.totalmem();
                const freeMem = os.freemem();
                const ramPct = ((totalMem - freeMem) / totalMem) * 100;
                let diskPct = 0;
                try {
                    const df = await execCmd("df -h / | awk 'NR==2 {print $5}'");
                    diskPct = parseInt(df.replace('%', ''));
                } catch {}

                if (ramPct > 90 || diskPct > 90) {
                    await sendAlert("ALERTA DE RECURSOS", `RAM: ${ramPct.toFixed(1)}% | Disco:
${diskPct}%`);
                    lastHwAlert = now;
                }
            } catch {}
        }
    }
```

2. MICROSSERVIÇO PROXY (Engine - Porta 6779)

Arquivo: server.ts

Servidor Dedicado ao Squid e Logs em Tempo Real

```
    import express from 'express';
    import cors from 'cors';
    import proxyRoutes from './routes/proxy-routes'; // Importando com o novo nome

    const app = express();
    const PORT = 6779;

    app.use(cors());
    app.use(express.json());

    // Logger
    app.use((req, res, next) => {
        console.log(`[PROXY-BACKEND] ${req.method} ${req.url}`);
        next();
    });

    // Rotas (Montadas em vrios prefixos para garantir que o Nginx acerte)
    app.use('/api/proxy', proxyRoutes);
    app.use('/proxy', proxyRoutes);
    app.use('/', proxyRoutes); // Fallback para /rules, etc

    // Certificado Download
    app.get('/api/cert/download', (req, res) => {
        res.download('/root/certificado_becker_proxy.der', 'certificado.der');
    });

    app.listen(PORT, '0.0.0.0', () => console.log(`>>> PROXY BACKEND rodando na porta ${PORT}`));
```

Arquivo: proxy-routes.ts

Rotas Renomeadas (Proxy Routes)

```
    import { Router } from 'express';
    import { execCmd } from '../utils/sys';
    import { Pool } from 'pg';
    import fs from 'fs';

    const router = Router();
    const pool = new Pool({ connectionString: 'postgres://postgres:becker_admin_secure@localhost:5432/
controlebeckercorp_v8' });

    const CMD_CTL = "sudo /bin/systemctl";

    // --- SYNC (Sincroniza DB -> Arquivos Texto do Squid) ---
    const syncSystem = async () => {
        try {
            // VIPs
            const vips = await pool.query("SELECT ip, description FROM proxy_vips");
            const vipList = vips.rows.map(r => `${r.ip} # ${r.description}`).join('\n');
            fs.writeFileSync('/etc/squid/vips.acl', vipList);

            // Reload Squid
            await execCmd(`${CMD_CTL} reload squid`);
        } catch (e) { console.error("Sync Error", e); }
    };

    // --- ENDPOINTS ---

    // Status
    router.get('/status', async (req, res) => {
        const status = await execCmd(`${CMD_CTL} is-active squid || echo inactive`);
        res.json({ status: status.trim() === 'active' ? 'running' : 'stopped' });
    });

    // Clientes Ativos (Baseado nos logs recentes do banco)
    router.get('/active-clients', async (req, res) => {
        try {
            // Pega os IPs distintos dos ltimos 10 minutos
            const r = await pool.query("SELECT DISTINCT client_ip FROM proxy_audit_log WHERE timestamp
> NOW() - INTERVAL '10 minutes' LIMIT 50");
            res.json(r.rows);
```

```
        } catch (e) { res.json([]); }
    });

    // Logs (Auditoria do Banco)
    router.get('/logs', async (req, res) => {
        try {
            const r = await pool.query("SELECT * FROM proxy_audit_log ORDER BY timestamp DESC LIMIT
100");
            res.json(r.rows);
        } catch (e) { res.json([]); }
    });

    // Interfaces
    router.get('/interfaces', async (req, res) => {
        try {
            const ifaces = fs.readdirSync('/sys/class/net').map(name => {
                let type = 'ethernet';
                if (name.includes('vlan') || name.includes('.')) type = 'vlan';
                const operstate = fs.existsSync(`/sys/class/net/${name}/operstate`)
                    ? fs.readFileSync(`/sys/class/net/${name}/operstate`, 'utf8').trim()
                    : 'down';
                return { name, label: name, type, enabled: operstate === 'up' };
            });
            res.json(ifaces);
        } catch(e) { res.json([]); }
    });

    // VIPs - Listar
    router.get('/vips', async (req, res) => {
        try {
            const r = await pool.query("SELECT * FROM proxy_vips ORDER BY id DESC");
            res.json(r.rows);
        } catch (e) { res.json([]); }
    });

    // VIPs - Adicionar
    router.post('/vips', async (req, res) => {
        await pool.query("INSERT INTO proxy_vips (ip, description) VALUES ($1, $2) ON CONFLICT (ip) DO
NOTHING", [req.body.ip, req.body.desc]);
        await syncSystem();
        res.json({ success: true });
    });

    // Arquivos de Regras (Flat Files)
    const RULES: any = {
        bloqueados: '/etc/squid/bloqueados.acl',
        permitidos: '/etc/squid/permitidos.acl',
        bancos: '/etc/squid/splice_whitelist.acl'
    };

    // Garante que arquivos existem
    Object.values(RULES).forEach((f:any) => {
        if (!fs.existsSync(f)) fs.writeFileSync(f, '');
    });

    router.get('/rules/:type', (req, res) => {
        const f = RULES[req.params.type];
        if (f && fs.existsSync(f)) res.json(fs.readFileSync(f, 'utf8').split('\n').filter(l =>
l.trim()));
        else res.json([]);
    });

    router.post('/rules/:type', (req, res) => {
        const f = RULES[req.params.type];
        if (f) {
            fs.writeFileSync(f, req.body.domains.join('\n'));
            execCmd(`${CMD_CTL} reload squid`);
            res.json({ success: true });
        } else res.status(400).json({});
    });

    export default router;
```

Arquivo: ingester.ts

Robô Ingestor de Logs (Tail -> Postgres)

```
    import { spawn } from 'child_process';
    import { Pool } from 'pg';
    import fs from 'fs';

    const pool = new Pool({ connectionString: 'postgres://postgres:becker_admin_secure@localhost:5432/
controlebeckercorp_v8' });
```

```
    const LOG_FILE = '/var/log/squid/access.log';

    console.log(`>>> [INGESTER] Monitorando ${LOG_FILE}...`);

    // Garante arquivo
    if (!fs.existsSync(LOG_FILE)) {
        fs.mkdirSync('/var/log/squid', { recursive: true });
        fs.writeFileSync(LOG_FILE, '');
    }

    const tail = spawn('tail', ['-F', '-n', '0', LOG_FILE]);

    tail.stdout.on('data', async (data) => {
        const lines = data.toString().split('\n');
        for (const line of lines) {
            if (!line.trim()) continue;
            try {
                // Formato Squid Customizado ou Padro
                // Tentativa de parsing genrico robusto
                const parts = line.split(/\s+/); // Separa por espaos

                // Ignora linhas curtas
                if (parts.length < 5) continue;

                const timestamp = new Date();

                // Tenta achar IP e URL com base na posio comum
                // Formato comum: time elapsed remotehost code/status bytes method URL rfc931
peerstatus/peerhost type
                const client_ip = parts[2] || '0.0.0.0';
                const status_raw = parts[3] || 'ERR/000';
                const bytes = parseInt(parts[4]) || 0;
                const method = parts[5] || '-';
                const url = parts[6] || '-';
                const username = parts[7] || '-';

                const action = status_raw.split('/')[0];
                const status_code = parseInt(status_raw.split('/')[1]) || 0;

                await pool.query(
                    'INSERT INTO proxy_audit_log (timestamp, client_ip, username, url, method,
status_code, bytes, action) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)',
                    [timestamp, client_ip, username, url.substring(0, 255), method, status_code,
bytes, action]
                );
            } catch (e) {
                // Silncio para no floodar console
            }
        }
    });
```

3. FRONTEND (React V8)

Arquivo: Proxy.jsx

Interface de Gestão do Proxy

```
    import React, { useState, useEffect } from 'react';
    import {
      Eye, User, List, Shield, Plus, Search, Trash2, Power,
      AlertTriangle, Crown, Network, Box, Router, Printer,
      Zap, Filter, Calendar, Download, ShieldAlert, ShieldCheck, FileKey,
      FolderOpen
    } from 'lucide-react';
    import { apiProxy } from '../services/apiProxy';
    import jsPDF from "jspdf";
    import autoTable from "jspdf-autotable";

    // Ajuste para caminho relativo (Nginx resolve)
    const API_RULES_URL = "/api/proxy";

    const TermoModal = ({ isOpen, onClose, onConfirm, user }) => {
        if (!isOpen) return null;
        const [checked, setChecked] = useState(false);
        return (
            <div className="fixed inset-0 bg-black/95 z-50 flex items-center justify-center p-4
backdrop-blur-sm">
                <div className="bg-white rounded-xl w-full max-w-4xl shadow-2xl flex flex-col h-
[85vh]">
                    <div className="bg-slate-900 p-6 flex justify-between items-center border-b-4
border-yellow-500">
                        <div><h2 className="text-xl font-black text-white uppercase tracking-
widest">Prefeitura de Jacarezinho</h2><p className="text-yellow-500 text-xs font-bold mt-1">SISTURS -
Tecnologia da Informao</p></div>
                        <Shield size={32} className="text-white opacity-20"/>
                    </div>
                    <div className="flex-1 p-10 overflow-y-auto bg-slate-50 font-serif text-slate-800
text-sm leading-relaxed text-justify">
                        <h3 className="text-center font-bold text-lg mb-8 uppercase underline">Termo
de Responsabilidade</h3>
                        <p className="mb-4">Eu, <strong>{user?.toUpperCase()}</strong>, declaro cincia
das normas:</p>
                        <div className="space-y-4">
                            <div className="p-4 bg-blue-50 border-l-4 border-blue-500 rounded-
r"><strong>1. MONITORAMENTO (LGPD):</strong> Todo trfego  auditado.</div>
                            <div className="p-4 bg-red-50 border-l-4 border-red-500 rounded-
r"><strong>2. PROIBIES:</strong> Contedo ilcito e jogos so vetados.</div>
                        </div>
                    </div>
                    <div className="bg-slate-100 p-6 border-t border-slate-200 flex justify-between
items-center">
                        <label className="flex items-center gap-2 font-bold text-xs"><input
type="checkbox" checked={checked} onChange={e=>setChecked(e.target.checked)}/> Li e ACEITO.</label>
                        <div className="flex gap-2"><button onClick={onClose} className="px-6 py-2 bg-
slate-300 rounded font-bold text-xs">CANCELAR</button><button onClick={onConfirm} disabled={!checked}
className={`px-6 py-2 rounded font-bold text-white text-xs ${checked?'bg-blue-800':'bg-slate-400'}`}
>CONFIRMAR</button></div>
                    </div>
                </div>
            </div>
        );
    };

    const ProxyPage = () => {
        const [activeTab, setActiveTab] = useState('networks');
        const [data, setData] = useState({
            users: [], vips: [], ifaces: [], logs: [], clients: [], status: 'checking',
            blockedRules: [], allowedRules: [], bankRules: []
        });
        const [form, setForm] = useState({ u:'', p:'', vipIp:'', vipDesc:'', newRule: '' });
        const [ui, setUi] = useState({ modal: false, loading: false, filter: '', selectedIp: 'all' });

        const loadAll = () => {
            apiProxy.get('/proxy/status').then(d => setData(p=>({...p, status:
d.data?.status||'error'}))).catch(()=>{});
            apiProxy.get('/proxy/interfaces').then(d => setData(p=>({...p, ifaces:
Array.isArray(d.data)?d.data:[]}))).catch(()=>{});
            apiProxy.get('/proxy/users').then(d => setData(p=>({...p, users: Array.isArray(d.data)?
d.data:[]}))).catch(()=>{});
            apiProxy.get('/proxy/vips').then(d => setData(p=>({...p, vips: Array.isArray(d.data)?d.data:
[]}))).catch(()=>{});
```

```
            apiProxy.get('/proxy/active-clients').then(d => setData(p=>({...p, clients:
Array.isArray(d.data)?d.data:[]}))).catch(()=>{});

            const logQuery = ui.selectedIp !== 'all' ? `?ip=${ui.selectedIp}` : '';
            apiProxy.get(`/proxy/logs${logQuery}`).then(d => setData(p=>({...p, logs:
Array.isArray(d.data)?d.data:[]}))).catch(()=>{});
        };

        const loadRules = async () => {
            try {
                const [blocked, allowed, banks] = await Promise.all([
                    fetch(`${API_RULES_URL}/rules/bloqueados`).then(r => r.json()),
                    fetch(`${API_RULES_URL}/rules/permitidos`).then(r => r.json()),
                    fetch(`${API_RULES_URL}/rules/bancos`).then(r => r.json())
                ]);
                setData(p => ({
                    ...p,
                    blockedRules: Array.isArray(blocked) ? blocked : [],
                    allowedRules: Array.isArray(allowed) ? allowed : [],
                    bankRules: Array.isArray(banks) ? banks : []
                }));
            } catch (e) { console.error(e); }
        };

        useEffect(() => { loadAll(); loadRules(); }, [ui.selectedIp]);
        useEffect(() => { const i = setInterval(loadAll, 5000); return () => clearInterval(i); },
[ui.selectedIp]);

        const handleUser = async () => { setUi(p=>({...p,loading:true})); await apiProxy.post('/proxy/
users',{username:form.u, password:form.p}); setForm({...form,u:'',p:''});
setUi({modal:false,loading:false,filter:''}); loadAll(); };
        const delUser = async (u) => { if(confirm('Excluir?')) await apiProxy.post('/proxy/users/
delete',{username:u}); loadAll(); };
        const handleVip = async (e) => { e.preventDefault(); if(form.vipIp) { await apiProxy.post('/
proxy/vips',{ip:form.vipIp, desc:form.vipDesc}); setForm({...form,vipIp:'',vipDesc:''}); loadAll(); }};
        const delVip = async (id) => { if(confirm('Excluir VIP?')) await apiProxy.post('/proxy/vips/
delete',{id}); loadAll(); };
        const toggleIface = async (iface, enabled) => { await apiProxy.post('/proxy/interfaces/toggle',
{iface, enabled}); loadAll(); };
        const handlePanic = async (action) => {
            if(action === 'activate' && !confirm("PARAR INTERCEPTAO?")) return;
            setUi(p=>({...p, loading:true})); await apiProxy.post('/proxy/panic', { action });
setTimeout(() => { loadAll(); setUi(p=>({...p, loading:false})); }, 3000);
        };

        const handleAddRule = async (type) => {
            if (!form.newRule) return;
            // Adiciona regra nova mantendo a lista atual
            const currentList = type === 'bloqueados' ? data.blockedRules : (type === 'permitidos' ?
data.allowedRules : data.bankRules);
            const newList = [...currentList, form.newRule.toLowerCase()];
            await saveRules(type, newList);
            setForm({...form, newRule: ''});
        };

        const handleDelRule = async (type, ruleToDelete) => {
            if(!confirm(`Remover regra: ${ruleToDelete}?`)) return;
            const currentList = type === 'bloqueados' ? data.blockedRules : (type === 'permitidos' ?
data.allowedRules : data.bankRules);
            const newList = currentList.filter(r => r !== ruleToDelete);
            await saveRules(type, newList);
        };

        const saveRules = async (type, domains) => {
            setUi(p=>({...p, loading:true}));
            try {
                await fetch(`${API_RULES_URL}/rules/${type}`, {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ domains })
                });
                await loadRules();
            } catch(e) { alert('Erro ao salvar regra'); }
            setUi(p=>({...p, loading:false}));
        };

        const downloadCert = () => { window.location.href = `${API_RULES_URL}/cert/download`; };

        const exportPDF = () => {
            if(!data.logs.length) return alert("Sem dados.");
            const doc = new jsPDF();
```

```
            doc.setFillColor(15,23,42); doc.rect(0,0,210,30,'F');
            doc.setTextColor(255); doc.setFontSize(14); doc.text("PREFEITURA DE JACAREZINHO",105,18,
{align:"center"});
            doc.setTextColor(234, 179, 8); doc.setFontSize(10); doc.text(`RELATRIO DE ACESSO`,105,25,
{align:"center"});

            const rows =
data.logs.filter(l=>JSON.stringify(l).toLowerCase().includes(ui.filter.toLowerCase())).map(l=>[
                new Date(l.timestamp).toLocaleDateString('pt-BR'),
                new Date(l.timestamp).toLocaleTimeString('pt-BR'),
                l.client_ip,
                l.url.substring(0,50),
                l.action
            ]);
            autoTable(doc, { startY:40, head:[['DATA', 'HORA','IP','SITE','STATUS']], body:rows, theme:
'grid' });
            doc.save('sisturs_auditoria.pdf');
        };

        return (
            <div className="space-y-8 pb-20">
                {/* Header */}
                <div className="flex flex-col md:flex-row justify-between items-center gap-4">
                    <div>
                        <h2 className="text-4xl font-black text-white italic tracking-tighter
uppercase flex items-center gap-3">
                            <Eye className="text-blue-600"/> PROXY <span className="text-
blue-600">CENTER</span>
                        </h2>
                        <p className="text-slate-500 text-sm mt-2 font-mono">SISTURS  Status: <span
className={data.status==='running'?'text-emerald-500':'text-red-500'}>{data.status.toUpperCase()}</
span></p>
                    </div>
                    <div className="flex flex-wrap gap-2 bg-slate-900 p-1 rounded-xl border border-
slate-800">
                        {[
                            {id:'networks', l:'Redes', i:Network},
                            {id:'rules', l:'Regras', i:ShieldAlert},
                            {id:'ssl', l:'Bancos/SSL', i:ShieldCheck},
                            {id:'users', l:'Usurios', i:User},
                            {id:'vips', l:'VIPs', i:Crown},
                            {id:'logs', l:'Auditoria', i:List}
                        ].map(t=>(
                            <button key={t.id} onClick={()=>setActiveTab(t.id)} className={`px-4 py-2
rounded-lg text-xs font-black uppercase flex gap-2 ${activeTab===t.id?'bg-blue-600 text-white':'text-
slate-500 hover:text-white'}`}>
                                <t.i size={14}/> {t.l}
                            </button>
                        ))}
                        <button onClick={()=>setActiveTab('panic')} className={`px-4 py-2 rounded-lg
text-xs font-black uppercase flex gap-2 ${activeTab==='panic'?'bg-red-600 text-white animate-
pulse':'text-red-500 hover:bg-red-950'}`}>
                            <AlertTriangle size={14}/> PNICO
                        </button>
                    </div>
                    <button onClick={downloadCert} className="bg-emerald-600 hover:bg-emerald-500 text-
white px-4 py-2 rounded-lg flex items-center gap-2 text-xs font-bold shadow-lg transition-all">
                        <Download size={16} /> CERTIFICADO .DER
                    </button>
                </div>

                {/* ABA REDES */}
                {activeTab === 'networks' && (
                    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6">
                        {data.ifaces.map(i => (
                            <div key={i.name} className={`bg-slate-900 border p-5 rounded-2xl flex flex-
col justify-between relative overflow-hidden group ${i.enabled ? 'border-blue-500/50 shadow-lg' :
'border-slate-800 opacity-70'}`}>
                                <div className="flex justify-between items-start mb-4 relative
z-10"><div className="flex items-center gap-3"><div className={`p-2 rounded-lg ${i.enabled ? 'bg-
blue-500/20 text-blue-400' : 'bg-slate-950 text-slate-600'}`}>{i.type==='docker'?<Box size={20}/
>:i.type==='vpn'?<Lock size={20}/>:<Router size={20}/>}</div><div><h4 className="text-white font-bold
text-xs uppercase">{i.label}</h4><p className="text-[10px] text-slate-500 font-mono">{i.name}</p></
div></div></div>
                                <button onClick={()=>toggleIface(i.name, !i.enabled)}
className={`relative z-10 w-full py-2 rounded-lg font-black uppercase text-[10px] flex items-center
justify-center gap-2 ${i.enabled ? 'bg-blue-600 hover:bg-blue-500 text-white' : 'bg-slate-800 text-
slate-500'}`}><Power size={12}/> {i.enabled ? 'ATIVO' : 'OFF'}</button>
                            </div>
                        ))}
                    </div>
```

```
                )}

                {/* ABA REGRAS (BLOQUEIOS) */}
                {activeTab === 'rules' && (
                    <div className="grid grid-cols-1 lg:grid-cols-2 gap-8">
                        {/* Lista Negra */}
                        <div className="bg-red-900/10 border border-red-500/20 p-6 rounded-3xl h-fit">
                            <div className="flex items-center gap-4 mb-4"><div className="p-3 bg-
red-500/20 text-red-500 rounded-2xl"><ShieldAlert size={32}/></div><div><h3 className="text-xl font-
black text-red-500 italic uppercase">SITES BLOQUEADOS</h3><p className="text-red-200/60 text-
xs">Bloqueio total imediato.</p></div></div>
                            <div className="flex gap-2 mb-4"><input value={form.newRule}
onChange={e=>setForm({...form, newRule:e.target.value})} placeholder="Ex: facebook.com"
className="flex-1 bg-slate-900 border border-slate-800 p-3 rounded-xl text-white outline-none"/><button
onClick={()=>handleAddRule('bloqueados')} className="bg-red-600 hover:bg-red-500 text-white px-4
rounded-xl"><Plus size={20}/></button></div>
                            <div className="bg-slate-900 border border-slate-800 rounded-xl p-2 max-h-
[300px] overflow-y-auto">{data.blockedRules.map((site, i) => (<div key={i} className="flex justify-
between items-center p-3 border-b border-slate-800 last:border-0 hover:bg-slate-800/50"><span
className="text-red-300 font-mono text-sm">{site}</span><button
onClick={()=>handleDelRule('bloqueados', site)} className="text-slate-600 hover:text-red-500"><Trash2
size={16}/></button></div>))}</div>
                        </div>
                        {/* Lista Branca */}
                        <div className="bg-emerald-900/10 border border-emerald-500/20 p-6 rounded-3xl
h-fit">
                            <div className="flex items-center gap-4 mb-4"><div className="p-3 bg-
emerald-500/20 text-emerald-500 rounded-2xl"><ShieldCheck size={32}/></div><div><h3 className="text-xl
font-black text-emerald-500 italic uppercase">PERMISSO VIP</h3><p className="text-emerald-200/60 text-
xs">Sempre liberado.</p></div></div>
                            <div className="flex gap-2 mb-4"><input value={form.newRule}
onChange={e=>setForm({...form, newRule:e.target.value})} placeholder="Ex: portal.gov.br"
className="flex-1 bg-slate-900 border border-slate-800 p-3 rounded-xl text-white outline-none"/><button
onClick={()=>handleAddRule('permitidos')} className="bg-emerald-600 hover:bg-emerald-500 text-white
px-4 rounded-xl"><Plus size={20}/></button></div>
                            <div className="bg-slate-900 border border-slate-800 rounded-xl p-2 max-h-
[300px] overflow-y-auto">{data.allowedRules.map((site, i) => (<div key={i} className="flex justify-
between items-center p-3 border-b border-slate-800 last:border-0 hover:bg-slate-800/50"><span
className="text-emerald-300 font-mono text-sm">{site}</span><button
onClick={()=>handleDelRule('permitidos', site)} className="text-slate-600 hover:text-red-500"><Trash2
size={16}/></button></div>))}</div>
                        </div>
                    </div>
                )}

                {/* ABA BANCOS / SSL (COM CATEGORIAS) */}
                {activeTab === 'ssl' && (
                    <div className="bg-slate-900 border border-slate-800 p-8 rounded-[40px]">
                        <div className="flex justify-between items-start mb-6">
                            <div className="flex items-center gap-4">
                                <div className="p-3 bg-blue-500/20 text-blue-400 rounded-2xl"><FileKey
size={32}/></div>
                                <div><h3 className="text-xl font-black text-white italic
uppercase">EXCEES SSL / BANCOS</h3><p className="text-slate-400 text-xs mt-1 max-w-lg">Domnios que
<strong>NO PODEM</strong> ser inspecionados (Splice). Adicione bancos e apps que do erro de
certificado.</p></div>
                            </div>
                        </div>

                        <div className="flex gap-4 mb-8">
                             <input value={form.newRule} onChange={e=>setForm({...form,
newRule:e.target.value})} placeholder="Ex: .caixa.gov.br (use # para Titulos)" className="flex-1 bg-
slate-950 border border-slate-800 p-4 rounded-xl text-white outline-none font-mono"/>
                             <button onClick={()=>handleAddRule('bancos')} className="bg-blue-600
hover:bg-blue-500 text-white px-8 rounded-xl font-black uppercase">ADICIONAR</button>
                        </div>

                        {/* RENDERIZAO INTELIGENTE: TTULOS vs ITENS */}
                        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
                            {data.bankRules.map((site, i) => {
                                // Se comear com #,  um Ttulo/Comentrio
                                if (site.trim().startsWith('#')) {
                                    return (
                                        <div key={i} className="col-span-full mt-4 pb-2 border-b
border-slate-800 flex items-center gap-2">
                                            <FolderOpen size={16} className="text-yellow-500"/>
                                            <span className="text-yellow-500 font-bold uppercase text-
xs tracking-widest">{site.replace(/#/g, '').trim()}</span>
                                        </div>
                                    );
                                }
```

```
                                // Se for site normal
                                return (
                                    <div key={i} className="bg-slate-950 p-4 rounded-xl border border-
slate-800 flex justify-between items-center group hover:border-blue-500/30">
                                        <span className="text-blue-200 font-mono font-bold text-
sm">{site}</span>
                                        <button onClick={()=>handleDelRule('bancos', site)}
className="text-slate-600 hover:text-red-500 opacity-50 group-hover:opacity-100"><Trash2 size={16}/></
button>
                                    </div>
                                );
                            })}
                        </div>
                    </div>
                )}

                {/* ABA PNICO */}
                {activeTab === 'panic' && (<div className="flex justify-center items-center h-
[400px]"><div className="bg-slate-900 border-2 border-red-600/50 p-10 rounded-3xl max-w-2xl w-full text-
center shadow-2xl relative overflow-hidden"><div className="absolute inset-0 bg-red-600/5 pointer-
events-none"/><AlertTriangle size={64} className="text-red-500 mx-auto mb-6"/><h2 className="text-3xl
font-black text-white uppercase mb-2">Controle de Emergncia</h2><p className="text-slate-400 mb-8 text-
sm">Utilize esta rea apenas se a interceptao estiver causando instabilidade.</p><div className="flex
gap-6 justify-center"><button onClick={()=>handlePanic('activate')} disabled={ui.loading ||
data.status==='stopped'} className={`flex-1 py-4 px-6 rounded-xl font-black uppercase text-sm shadow-lg
flex items-center justify-center gap-3 ${data.status==='stopped' ? 'bg-slate-800 text-slate-500 cursor-
not-allowed' : 'bg-red-600 hover:bg-red-500 text-white'}`}><Power size={20}/> PARAR TUDO</
button><button onClick={()=>handlePanic('restore')} disabled={ui.loading || data.status==='running'}
className={`flex-1 py-4 px-6 rounded-xl font-black uppercase text-sm shadow-lg flex items-center
justify-center gap-3 ${data.status==='running' ? 'bg-slate-800 text-slate-500 cursor-not-allowed' : 'bg-
emerald-600 hover:bg-emerald-500 text-white'}`}><Zap size={20}/> RESTAURAR SISTEMA</button></div><p
className="mt-6 text-xs text-slate-500 font-mono">Status Atual: {data.status === 'running' ? <span
className="text-emerald-500">OPERACIONAL</span> : <span className="text-red-500">PARADO</span>}</p></
div></div>)}

                {/* ABA USURIOS */}
                {activeTab === 'users' && <div className="grid grid-cols-1 lg:grid-cols-2 gap-8"><div
className="bg-slate-900 border border-slate-800 p-8 rounded-[40px] h-fit"><h3 className="text-white
font-bold uppercase mb-4 flex gap-2"><Plus size={16} className="text-emerald-500"/> Cadastro</h3><div
className="space-y-4"><input value={form.u} onChange={e=>setForm({...form,u:e.target.value})}
placeholder="Usurio" className="w-full bg-slate-950 border border-slate-800 p-3 rounded-xl text-white
outline-none"/><input value={form.p} onChange={e=>setForm({...form,p:e.target.value})}
placeholder="Senha" type="password" className="w-full bg-slate-950 border border-slate-800 p-3 rounded-
xl text-white outline-none"/><button onClick={()=>{if(form.u&&form.p) setUi({...ui,modal:true})}}
className="w-full bg-emerald-600 hover:bg-emerald-500 text-white py-3 rounded-xl font-black
uppercase">INICIAR CADASTRO</button></div></div><div className="space-y-2 max-h-[500px] overflow-y-
auto">{data.users.map(u => (<div key={u.id} className="bg-slate-900 border border-slate-800 p-3 rounded-
xl flex justify-between items-center"><div className="flex items-center gap-3"><div className="bg-
slate-950 p-2 rounded-lg text-blue-500"><User size={16}/></div><span className="text-white font-bold
text-sm">{u.username}</span></div><button onClick={()=>delUser(u.username)} className="text-slate-600
hover:text-red-500"><Trash2 size={16}/></button></div>))}</div></div>}

                {/* ABA VIPS */}
                {activeTab === 'vips' && <div className="grid grid-cols-1 lg:grid-cols-2 gap-8"><div
className="bg-yellow-900/10 border border-yellow-500/20 p-6 rounded-3xl h-fit"><div className="flex
items-center gap-4 mb-4"><div className="p-3 bg-yellow-500/20 text-yellow-500 rounded-2xl"><Crown
size={32}/></div><div><h3 className="text-xl font-black text-yellow-500 italic uppercase">VIP BYPASS</
h3><p className="text-yellow-200/60 text-xs">Acesso Total.</p></div></div><div className="space-
y-3"><input value={form.vipIp} onChange={e=>setForm({...form,vipIp:e.target.value})} placeholder="IP"
className="w-full bg-slate-900 border border-slate-800 p-3 rounded-xl text-white outline-none font-
mono"/><input value={form.vipDesc} onChange={e=>setForm({...form,vipDesc:e.target.value})}
placeholder="Descrio" className="w-full bg-slate-900 border border-slate-800 p-3 rounded-xl text-white
outline-none"/><button onClick={handleVip} className="w-full bg-yellow-600 hover:bg-yellow-500 text-
black font-black uppercase py-3 rounded-xl">ADD VIP</button></div></div><div className="bg-slate-900
border border-slate-800 rounded-[30px] p-6 h-fit"><h4 className="text-slate-500 font-bold uppercase
text-xs mb-4">Lista VIP</h4><div className="space-y-2">{data.vips.map(v => (<div key={v.id}
className="bg-slate-950 p-4 rounded-2xl border border-slate-800 flex justify-between items-center group
hover:border-yellow-500/30"><div><span className="text-white font-mono font-bold block">{v.ip}</
span><span className="text-slate-500 text-[10px] uppercase">{v.description}</span></div><button
onClick={()=>delVip(v.id)} className="text-slate-600 hover:text-red-500"><Trash2 size={18}/></button></
div>))}</div></div></div>}

                {/* ABA LOGS */}
                {activeTab === 'logs' && <div className="space-y-4"><div className="flex flex-col
md:flex-row justify-between items-center gap-4"><div className="flex gap-3 w-full md:w-auto"><div
className="flex items-center gap-2 bg-slate-900 p-2 rounded-xl border border-slate-800"><Filter
size={16} className="text-blue-500 ml-2"/><select value={ui.selectedIp} onChange={e=>setUi({...ui,
selectedIp: e.target.value})} className="bg-transparent text-white text-xs outline-none w-32 md:w-48
font-mono"><option value="all" className="bg-slate-900 text-slate-400">TODOS OS CLIENTES</
option>{data.clients.map(c => (<option key={c.client_ip} value={c.client_ip} className="bg-slate-900
text-white">{c.client_ip}</option>))}</select></div><div className="flex items-center gap-2 bg-
```

```
slate-900 p-2 rounded-xl border border-slate-800 flex-1"><Search size={16} className="text-slate-500
ml-2"/><input value={ui.filter} onChange={e=>setUi({...ui,filter:e.target.value})} placeholder="Busca
Rpida" className="bg-transparent text-white text-xs w-full outline-none"/></div></div><button
onClick={exportPDF} className="bg-white hover:bg-slate-200 text-slate-900 px-6 py-2 rounded-xl font-
black uppercase text-xs flex items-center gap-2 shadow-lg w-full md:w-auto justify-center"><Printer
size={16}/> PDF</button></div><div className="bg-slate-900 border border-slate-800 rounded-2xl overflow-
hidden shadow-xl"><div className="overflow-x-auto"><table className="w-full text-left text-xs font-
mono"><thead className="bg-slate-950 text-slate-500 uppercase"><tr><th className="p-3"><div
className="flex items-center gap-1"><Calendar size={10}/> Data</div></th><th className="p-3">Hora</
th><th className="p-3">IP Cliente</th><th className="p-3">Site / Recurso</th><th
className="p-3">Status</th></tr></thead><tbody>{data.logs.filter(l=>JSON.stringify(l).toLowerCase().incl
udes(ui.filter.toLowerCase())).map((l,i)=>(<tr key={i} className="border-b border-slate-800/50 hover:bg-
slate-800/30"><td className="p-3 text-slate-500 font-bold">{new
Date(l.timestamp).toLocaleDateString('pt-BR')}</td><td className="p-3 text-slate-400">{new
Date(l.timestamp).toLocaleTimeString('pt-BR')}</td><td className="p-3 text-blue-400 font-
bold">{l.client_ip}</td><td className="p-3 text-slate-300 truncate max-w-xs" title={l.url}
>{l.url.substring(0,60)}</td><td className="p-3"><span className={`px-2 py-0.5 rounded text-[9px] font-
black ${l.action.includes('DENIED')?'bg-red-500/20 text-red-500':'bg-green-500/20 text-green-500'}`}
>{l.action}</span></td></tr>))}</tbody></table>{data.logs.length === 0 && <div className="p-10 text-
center text-slate-600 italic">Nenhum registro encontrado para este filtro.</div>}</div></div></div>}

                <TermoModal isOpen={ui.modal} onClose={()=>setUi({...ui,modal:false})}
onConfirm={handleUser} user={form.u} />
            </div>
        );
    };
    export default ProxyPage;
```

Arquivo: Control.jsx

Painel de Controle e Botão de Pânico

```
    import React, { useState, useEffect } from 'react';
    import { motion, AnimatePresence } from 'framer-motion';
    import { Shield, Globe, Router, Folder, FileText, Lock, Database, Terminal, Play, Square,
RefreshCw, AlertTriangle, Zap, Mail, Settings, CheckCircle, Send, FileArchive, Eye, Unlock, Flame,
Activity, Radio, Save, Trash2, X, Cpu } from 'lucide-react';
    import { api } from '../services/api';

    const icons = { Shield, Globe, Router, Folder, FileText, Lock, Database, Terminal };

    const MonitoringRadar = () => (
        <div className="relative flex items-center justify-center w-12 h-12 shrink-0">
            <div className="absolute w-full h-full bg-blue-500/20 rounded-full animate-ping
opacity-75"></div>
            <div className="absolute w-8 h-8 bg-blue-500/40 rounded-full animate-pulse"></div>
            <div className="relative z-10 bg-slate-900 border border-blue-500 p-2 rounded-full flex
items-center justify-center">
                <Radio className="text-blue-400 animate-[spin_3s_linear_infinite]" size={16}/>
            </div>
        </div>
    );

    const AiSentinelCard = ({ services }) => {
        const failedServices = services.filter(s => s.status === 'stopped' || s.status === 'error');
        const isCrisis = failedServices.length > 0;

        return (
            <motion.div
                initial={{ opacity: 0, y: -20 }}
                animate={{ opacity: 1, y: 0 }}
                className={`relative overflow-hidden rounded-[30px] border p-8 mb-8 transition-all
duration-500
                ${isCrisis
                    ? 'bg-red-950/40 border-red-500/50 shadow-[0_0_50px_rgba(220,38,38,0.2)]'
                    : 'bg-slate-900/80 border-blue-500/30 shadow-[0_0_30px_rgba(59,130,246,0.1)]'
                }`}
            >
                <div className="absolute inset-0 bg-[url('https://www.transparenttextures.com/patterns/
circuit-board.png')] opacity-5 pointer-events-none"/>
                {isCrisis && <div className="absolute inset-0 bg-red-600/10 animate-pulse pointer-
events-none"/>}
                {!isCrisis && <div className="absolute top-0 left-0 w-full h-1 bg-gradient-to-r from-
transparent via-blue-500 to-transparent animate-[shimmer_2s_infinite]"/>}

                <div className="relative z-10 flex flex-col md:flex-row items-center justify-between
gap-6">
                    <div className="flex items-center gap-6">
                        <div className={`relative flex items-center justify-center w-20 h-20 rounded-
full border-2 ${isCrisis ? 'border-red-500 bg-red-900/20' : 'border-blue-500 bg-blue-900/20'}`}>
                            <Cpu size={40} className={`relative z-10 ${isCrisis ? 'text-red-500 animate-
bounce' : 'text-blue-400'}`}/>
```

```
                        </div>
                        <div>
                            <h2 className={`text-2xl font-black italic uppercase tracking-wider
${isCrisis ? 'text-red-500' : 'text-blue-400'}`}>
                                {isCrisis ? 'ALERTA DE SEGURANA' : 'IA SENTINELA ATIVA'}
                            </h2>
                            <div className="flex items-center gap-2 mt-1">
                                <span className={`w-2 h-2 rounded-full ${isCrisis ? 'bg-red-500 animate-
ping' : 'bg-emerald-500 animate-pulse'}`}/>
                                <p className="text-xs font-mono text-slate-400 uppercase">
                                    {isCrisis ? 'FALHA CRTICA IDENTIFICADA' : 'MONITORAMENTO EM TEMPO
REAL'}
                                </p>
                            </div>
                        </div>
                    </div>
                    <div className="flex-1 w-full md:w-auto bg-slate-950/50 rounded-2xl p-4 border
border-slate-800 font-mono text-[10px] h-24 overflow-hidden relative">
                        <div className="space-y-1">
                            <p className="text-slate-500">&gt;&gt;&gt; INICIANDO ESCANEAMENTO...</p>
                            <p className="text-slate-500">&gt;&gt;&gt; ANALISANDO {services.length}
SERVIOS...</p>
                            {isCrisis ? (
                                failedServices.map((s, i) => (
                                    <p key={i} className="text-red-400 font-bold animate-
pulse">[ALERTA] FALHA EM {s.name.toUpperCase()} DETECTADA!</p>
                                ))
                            ) : (
                                 <><p className="text-emerald-500">&gt;&gt;&gt; TODOS SISTEMAS
OPERACIONAIS.</p></>
                            )}
                        </div>
                    </div>
                </div>
            </motion.div>
        );
    };

    const ServiceCard = ({ svc, onToggle, index }) => {
        const Icon = icons[svc.icon] || Globe;
        const isRunning = svc.status === 'running';
        const isError = svc.status === 'error';
        const [loading, setLoading] = useState(false);

        const handleAction = async (action) => {
            if(!confirm(`${action.toUpperCase()} ${svc.label}?`)) return;
            setLoading(true);
            await onToggle(svc.name, action);
            setLoading(false);
        };

        return (
            <motion.div
                initial={{ opacity: 0, y: 20 }}
                animate={{ opacity: 1, y: 0 }}
                transition={{ delay: index * 0.1 }}
                whileHover={{ scale: 1.02, y: -5 }}
                className={`bg-slate-900 border p-6 rounded-[30px] relative overflow-hidden group
shadow-lg transition-all
                ${isError ? 'border-red-500' : isRunning ? 'border-emerald-500/30' : 'border-
slate-800'}`}
            >
                <div className="relative z-10 flex justify-between items-start mb-6">
                    <div className="flex items-center gap-3">
                        <div className={`p-3 rounded-2xl transition-all duration-500 ${isError ? 'bg-
red-500/20 text-red-500' : isRunning ? 'bg-emerald-500/10 text-emerald-500' : 'bg-slate-800 text-
slate-500'}`}>
                            <div className={isRunning ? 'animate-[spin_10s_linear_infinite]' : ''}
><Icon size={24}/></div>
                        </div>
                        <div>
                            <h3 className="text-sm font-black text-white italic uppercase tracking-
wider">{svc.label}</h3>
                            <p className="text-[10px] text-slate-500 font-mono flex items-center gap-1">
                                {isRunning && <Activity size={10} className="text-emerald-500 animate-
bounce"/>} {svc.name}
                            </p>
                        </div>
                    </div>
                    <div className={`px-2 py-1 rounded text-[9px] font-black uppercase border flex
items-center gap-1 ${isError ? 'bg-red-500 text-white border-red-600' : isRunning ? 'bg-emerald-500/10
```

```
text-emerald-500 border-emerald-500/20' : 'bg-slate-800 text-slate-500 border-slate-700'}`}>
                        {loading ? '...' : svc.status}
                    </div>
                </div>
                <div className={`relative z-10 flex gap-2 transition-opacity duration-300
${isRunning ? 'opacity-40 group-hover:opacity-100' : 'opacity-100'}`}>
                    {!isRunning && <button onClick={()=>handleAction('start')} disabled={loading}
className="flex-1 py-2 bg-emerald-600 hover:bg-emerald-500 text-white rounded-xl text-xs font-black
uppercase flex items-center justify-center gap-2 shadow-lg"><Play size={12}/> Iniciar</button>}
                    {isRunning && (
                        <>
                            <button onClick={()=>handleAction('restart')} disabled={loading}
className="flex-1 py-2 bg-blue-600 hover:bg-blue-500 text-white rounded-xl text-xs font-black uppercase
flex items-center justify-center gap-2 shadow-lg"><RefreshCw size={12}/> Reiniciar</button>
                            <button onClick={()=>handleAction('stop')} disabled={loading}
className="py-2 px-3 bg-red-900/20 hover:bg-red-600 text-red-500 hover:text-white rounded-xl border
border-red-900/50 transition-all hover:rotate-90"><Square size={12}/></button>
                        </>
                    )}
                </div>
            </motion.div>
        );
    };

    const SmtpModal = ({ isOpen, onClose }) => {
        const [conf, setConf] = useState({ host: 'smtp.gmail.com', port: 587, user: '', pass: '', to:
'' });
        const [status, setStatus] = useState({ msg: '', type: '' });
        const [loading, setLoading] = useState(false);

        useEffect(() => {
            if(isOpen) {
                setStatus({ msg: '', type: '' });
                api.get('/api/control/smtp').then(d => { if(d?.data) setConf(d.data); }).catch(() =>
setStatus({ msg: 'Erro ao ler dados.', type: 'error' }));
            }
        }, [isOpen]);

        const test = async () => {
            setStatus({ msg: 'Enviando teste...', type: 'info' }); setLoading(true);
            try {
                const res = await api.post('/api/control/smtp/test', conf);
                if (res.data.success) setStatus({ msg: 'E-mail Enviado! ', type: 'success' });
                else throw new Error();
            } catch { setStatus({ msg: 'Falha no envio.', type: 'error' }); }
            setLoading(false);
        };

        const save = async () => {
            setStatus({ msg: 'Gravando...', type: 'info' }); setLoading(true);
            try {
                await api.post('/api/control/smtp', conf);
                setStatus({ msg: 'Configurao Salva! ', type: 'success' });
            } catch { setStatus({ msg: 'ERRO AO SALVAR!', type: 'error' }); }
            setLoading(false);
        };

        if (!isOpen) return null;

        return (
            <div className="fixed inset-0 bg-black/90 z-50 flex items-center justify-center p-4
backdrop-blur-sm">
                <motion.div initial={{scale:0.9, opacity:0}} animate={{scale:1, opacity:1}}
className="bg-slate-900 border border-slate-700 p-8 rounded-[30px] w-full max-w-lg shadow-2xl relative">
                    <button onClick={onClose} className="absolute top-6 right-6 text-slate-500
hover:text-white"><X size={20}/></button>
                    <h3 className="text-xl font-black text-white italic uppercase mb-6 flex items-
center gap-2"><div className="bg-blue-500/20 p-2 rounded-lg text-blue-500"><Mail size={20}/></
div>Configurao SMTP</h3>
                    <div className="grid grid-cols-1 md:grid-cols-2 gap-4 mb-4">
                        <div className="col-span-2"><label className="text-[10px] text-slate-500
uppercase font-bold ml-1">Servidor SMTP</label><input value={conf.host}
onChange={e=>setConf({...conf,host:e.target.value})} className="w-full bg-slate-950 border border-
slate-800 p-3 rounded-xl text-white text-xs outline-none focus:border-blue-500"/></div>
                        <div><label className="text-[10px] text-slate-500 uppercase font-bold
ml-1">Usurio (E-mail)</label><input value={conf.user}
onChange={e=>setConf({...conf,user:e.target.value})} className="w-full bg-slate-950 border border-
slate-800 p-3 rounded-xl text-white text-xs outline-none focus:border-blue-500"/></div>
                        <div><label className="text-[10px] text-slate-500 uppercase font-bold
ml-1">Senha (App Password)</label><input value={conf.pass} type="password"
onChange={e=>setConf({...conf,pass:e.target.value})} className="w-full bg-slate-950 border border-
```

```
slate-800 p-3 rounded-xl text-white text-xs outline-none focus:border-blue-500"/></div>
                        <div className="col-span-2 flex gap-4">
                            <div className="w-24"><label className="text-[10px] text-slate-500
uppercase font-bold ml-1">Porta</label><input value={conf.port}
onChange={e=>setConf({...conf,port:e.target.value})} className="w-full bg-slate-950 border border-
slate-800 p-3 rounded-xl text-white text-xs outline-none focus:border-blue-500"/></div>
                            <div className="flex-1"><label className="text-[10px] text-slate-500
uppercase font-bold ml-1">Destinatrio Alertas</label><input value={conf.to}
onChange={e=>setConf({...conf,to:e.target.value})} className="w-full bg-slate-950 border border-
slate-800 p-3 rounded-xl text-white text-xs outline-none focus:border-blue-500"/></div>
                        </div>
                    </div>
                    {status.msg && <div className={`text-xs p-3 rounded-xl text-center font-bold mb-6
flex items-center justify-center gap-2 ${status.type==='success'?'text-emerald-400 bg-emerald-900/20
border border-emerald-500/20':'text-red-400 bg-red-900/20 border border-red-500/20'}`}>{status.type ===
'success' ? <CheckCircle size={14}/> : <AlertTriangle size={14}/>} {status.msg}</div>}
                    <div className="flex gap-3 pt-4 border-t border-slate-800">
                        <button onClick={test} disabled={loading} className="flex-1 py-3 bg-slate-800
hover:bg-slate-700 text-white rounded-xl text-xs font-black flex justify-center gap-2 transition-
colors"><Send size={14}/> Testar</button>
                        <button onClick={save} disabled={loading} className="flex-1 py-3 bg-blue-600
hover:bg-blue-500 text-white rounded-xl text-xs font-black flex justify-center gap-2 transition-colors
shadow-lg shadow-blue-900/20"><Save size={14}/> Salvar</button>
                    </div>
                </motion.div>
            </div>
        );
    };

    const ControlPage=()=>{
        const[services,setServices]=useState([]);
        const[panicMode,setPanicMode]=useState(false);
        const[actionStatus,setActionStatus]=useState({loading:false,msg:''});
        const[auditData,setAuditData]=useState(null);
        const[showSmtp,setShowSmtp]=useState(false);

        const load=async()=>{
            try {
                const d=await api.get('/api/control/services');
                if(Array.isArray(d.data)) setServices(d.data);
            } catch {}
        };

        useEffect(()=>{load();const i=setInterval(load,3000);return()=>clearInterval(i)},[]);

        const toggle=async(name,action)=>{await api.post('/api/control/toggle',{name,action});load()};
        const runAudit=async()=>{setAuditData({loading:true});try{const res=await api.post('/api/
control/audit');setAuditData({...res.data,loading:false})}catch(e){setAuditData({error:true})}};
        const runTactical=async(action,label)=>{if(!confirm(`Confirmar: ${label}?
`))return;setActionStatus({loading:true,msg:`Executando: ${label}...`});try{const res=await api.post('/
api/control/tactical',{action});setActionStatus({loading:false,msg:res.data.message});load()}catch(e)
{setActionStatus({loading:false,msg:'Erro.'})}};
        const runFodeuTudo=async()=>{if(!confirm(" RESET
TOTAL?"))return;setActionStatus({loading:true,msg:'RESETANDO...'});try{await api.post('/api/control/pani
c');setActionStatus({loading:false,msg:'RESETADO.'});setTimeout(()=>{setActionStatus({loading:false,msg:
''});setPanicMode(false);load()},4000)}catch(e){setActionStatus({loading:false,msg:'ERRO.'})}};

        return(
            <div className="space-y-8 pb-20 relative">
                <div className="flex flex-col md:flex-row justify-between items-center gap-6">
                    <div className="flex items-center gap-6">
                        <MonitoringRadar/>
                        <div>
                            <h2 className="text-4xl font-black text-white italic tracking-tighter
uppercase flex items-center gap-3">
                                CENTRO DE <span className="text-blue-600">CONTROLE</span>
                            </h2>
                            <p className="text-slate-500 text-sm mt-2 flex items-center gap-2">
                                <span className="w-2 h-2 rounded-full bg-emerald-500 animate-pulse"/>
Infraestrutura Becker Corp V8
                            </p>
                        </div>
                    </div>
                    <div className="flex gap-3">
                        {/* BOTO MANTIDO COMO 'ALERTAS' MAS ABRE O MODAL SMTP */}
                        <button onClick={()=>setShowSmtp(true)} className="px-4 py-3 bg-slate-900
border border-slate-800 text-slate-400 hover:text-white rounded-xl flex items-center gap-2 text-xs font-
black uppercase transition-colors hover:border-blue-500 hover:text-blue-500">
                            <Settings size={16}/> Alertas
                        </button>
                        <button onClick={()=>setPanicMode(!panicMode)} className={`px-6 py-3 rounded-xl
```

```
flex items-center gap-2 text-xs font-black uppercase transition-all shadow-lg hover:scale-105
${panicMode?'bg-slate-800 text-slate-400':'bg-red-600 hover:bg-red-500 text-white shadow-red-900/20'}`}>
                            <AlertTriangle size={16}/> {panicMode?'Fechar rea de Risco':'rea de Risco'}
                        </button>
                    </div>
                </div>

                <div className="w-full h-[1px] bg-slate-800 overflow-hidden relative">
                    <div className="absolute top-0 left-0 w-1/3 h-full bg-gradient-to-r from-
transparent via-blue-500 to-transparent animate-[shimmer_3s_infinite]"/>
                </div>

                <AiSentinelCard services={services} />

                <AnimatePresence>
                    {panicMode&&(<motion.div initial={{opacity:0,height:0}}
animate={{opacity:1,height:'auto'}} exit={{opacity:0,height:0}} className="bg-red-950/30 border-2
border-red-600 border-dashed rounded-[30px] overflow-hidden relative mb-8"><div className="absolute
inset-0 bg-[url('https://www.transparenttextures.com/patterns/carbon-fibre.png')] opacity-10"/><div
className="p-8 relative z-10"><div className="flex justify-between items-center mb-8 border-b border-
red-900/50 pb-4"><h3 className="text-xl font-black text-red-500 italic uppercase flex items-center
gap-2"><Flame className="animate-pulse"/> Painel Ttico</h3>{actionStatus.msg&&<span className="text-xs
font-mono bg-black/50 px-3 py-1 rounded text-yellow-400 border border-yellow-500/30 animate-
pulse">{actionStatus.msg}</span>}</div><div className="grid grid-cols-1 lg:grid-cols-3 gap-8"><div
className="border-r border-red-900/50 pr-8"><h4 className="text-slate-400 font-bold uppercase text-xs
mb-4 flex items-center gap-2"><Eye size={14}/> Auditoria</h4><button onClick={runAudit} className="w-
full py-3 bg-slate-900 border border-slate-700 hover:bg-slate-800 text-white rounded-xl text-xs font-
black uppercase flex items-center justify-center gap-2 mb-4"><FileArchive size={14}/> Gerar & Ler
Snapshot</button>{auditData?.files&&<div className="bg-black/50 rounded-xl p-4 h-32 overflow-y-auto
custom-scrollbar border border-slate-800">{auditData.files.map((f,i)=><div key={i} className="text-
[9px] text-slate-300 font-mono border-b border-slate-800/50 py-0.5">{f}</div>)}</div>}</div><div
className="lg:col-span-2"><h4 className="text-slate-400 font-bold uppercase text-xs mb-4 flex items-
center gap-2"><Unlock size={14}/> Correes</h4><div className="grid grid-cols-1 md:grid-cols-2
gap-4"><button onClick={()=>runTactical('fail2ban_unlock','Desbloquear Fail2Ban')} className="p-4 bg-
slate-900 border border-slate-700 hover:bg-blue-900/30 hover:border-blue-500 text-white rounded-2xl
flex items-center gap-4 transition-all group"><div className="bg-slate-950 p-2 rounded-lg text-
blue-500"><Lock size={20}/></div><div className="text-left font-bold text-xs uppercase">Desbloquear
Fail2Ban</div></button><button onClick={()=>runTactical('firewall_reset','Restaurar Firewall')}
className="p-4 bg-slate-900 border border-slate-700 hover:bg-yellow-900/30 hover:border-yellow-500 text-
white rounded-2xl flex items-center gap-4 transition-all group"><div className="bg-slate-950 p-2
rounded-lg text-yellow-500"><Shield size={20}/></div><div className="text-left font-bold text-xs
uppercase">Restaurar Firewall</div></button><button onClick={()=>runTactical('dhcp_restart','Reiniciar
DHCP')} className="p-4 bg-slate-900 border border-slate-700 hover:bg-emerald-900/30 hover:border-
emerald-500 text-white rounded-2xl flex items-center gap-4 transition-all group"><div className="bg-
slate-950 p-2 rounded-lg text-emerald-500"><Router size={20}/></div><div className="text-left font-bold
text-xs uppercase">Reiniciar DHCP</div></button><button
onClick={()=>runTactical('db_restart','Reiniciar Banco')} className="p-4 bg-slate-900 border border-
slate-700 hover:bg-purple-900/30 hover:border-purple-500 text-white rounded-2xl flex items-center gap-4
transition-all group"><div className="bg-slate-950 p-2 rounded-lg text-purple-500"><Database size={20}/
></div><div className="text-left font-bold text-xs uppercase">Reiniciar Database</div></button></
div><div className="mt-8 pt-6 border-t border-red-900/30"><button onClick={runFodeuTudo} className="w-
full py-4 bg-red-900/20 hover:bg-red-600 border border-red-800 text-red-500 hover:text-white
rounded-2xl font-black uppercase text-sm tracking-widest transition-all shadow-lg flex items-center
justify-center gap-3"><Zap size={18}/> Protocolo "FODEU TUDO"</button></div></div></div></div></
motion.div>)}</AnimatePresence>

                <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 gap-6">
                    {services.map((svc,i)=>(<ServiceCard key={i} index={i} svc={svc} onToggle={toggle}/
>))}
                </div>
                <SmtpModal isOpen={showSmtp} onClose={()=>setShowSmtp(false)}/>
            </div>
        );
    };
    export default ControlPage;
```

4. SCRIPTS DE INFRAESTRUTURA

Arquivo: panic_on.sh

Protocolo Pânico (Ativar)

```
#!/bin/bash
echo ">>> [PNICO] Desativando Interceptao..."
# 1. Para o Squid
systemctl stop squid
# 2. Limpa regras de NAT (Redirecionamentos)
iptables -t nat -F PREROUTING
# 3. Garante NAT de Sada (Internet)
WAN_IFACE=$(ip route show | grep default | awk '{print $5}' | head -n 1)
iptables -t nat -F POSTROUTING
iptables -t nat -A POSTROUTING -o $WAN_IFACE -j MASQUERADE
# 4. Remove bloqueio QUIC (Libera tudo)
iptables -D FORWARD -p udp --dport 443 -j DROP 2>/dev/null || true
# Salva
netfilter-persistent save >/dev/null 2>&1 || true
echo ">>> [PNICO] Sistema em Bypass Total."
```

Arquivo: panic_off.sh

Protocolo Pânico (Desativar/Restaurar)

```
#!/bin/bash
echo ">>> [NETWORK] Aplicando Regras de Firewall (V2 - High Performance)..."

# 1. Habilita Roteamento
sysctl -w net.ipv4.ip_forward=1 >/dev/null

# 2. Limpa tabelas de NAT e Filtro (Reset limpo)
iptables -F
iptables -t nat -F
iptables -X
iptables -t nat -X

# 3. NAT de Sada (Internet para todos)
WAN_IFACE=$(ip route show | grep default | awk '{print $5}' | head -n 1)
iptables -t nat -A POSTROUTING -o $WAN_IFACE -j MASQUERADE

# 4. Bloqueio QUIC (Performance: REJECT rpido)
# Fora navegadores a usarem TCP imediatamente, evitando espera/lentido
iptables -A FORWARD -p udp --dport 443 -j REJECT --reject-with icmp-port-unreachable
iptables -A INPUT -p udp --dport 443 -j REJECT --reject-with icmp-port-unreachable

# 5. Interceptao Inteligente (A CORREO EST AQUI)
# Redireciona para o Squid (3128) APENAS se o destino NO for rede local.
# Sintaxe correta: ! -d 192.168.0.0/16
# Isso evita loops e garante que acesso a impressoras/servidores locais seja direto (rpido).

NETWORKS=("192.168.10.0/24" "192.168.30.0/24" "192.168.40.0/24" "192.168.50.0/24" "192.168.70.0/24")

for NET in "${NETWORKS[@]}"; do
    # HTTP (80) -> Squid (Exceto destino local)
    iptables -t nat -A PREROUTING -s $NET ! -d 192.168.0.0/16 -p tcp --dport 80 -j REDIRECT --to-
port 3128

    # HTTPS (443) -> Squid (Exceto destino local)
    iptables -t nat -A PREROUTING -s $NET ! -d 192.168.0.0/16 -p tcp --dport 443 -j REDIRECT --to-
port 3128
done

# Permite acesso ao Painel na porta nova
iptables -A INPUT -p tcp --dport 6779 -j ACCEPT

# Salva regras
if command -v netfilter-persistent &> /dev/null; then
    netfilter-persistent save >/dev/null 2>&1 || true
fi

echo ">>> [SISTEMA] Monitoramento Ativo e Firewall Corrigido."
```